AIAC-10.1

Name: Siri Vempati

Htno:2303A51309

Batchno:05

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)
```

Expected Output:

• Corrected and runnable Python code with explanations of the fixes.

Prompt:

Identify and fix all syntax and logic errors in the following Python script.
Return corrected and fully runnable Python code.
Also explain each fix clearly.

Code:

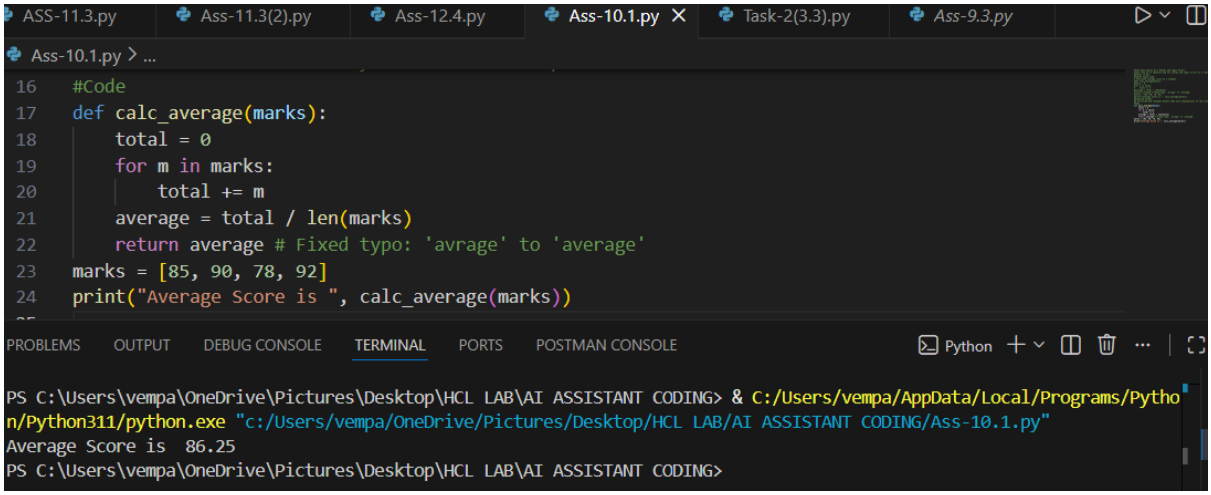
```
def calc_average(marks):

    total = 0

    for m in marks:

        total += m

    average = total / len(marks)
```

return average # Fixed typo: 'avrage' to 'average'

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))

Output:



Explanation:

The original code had a typo in the return statement where 'avrage' was misspelled.

The corrected version returns 'average' instead of 'avrage'.

Additionally, the original code was not properly indented, which would have caused a syntax error.

The corrected code is now properly indented and will run without errors, calculating and printing the average score of the student.

<u>Task Description #2 – PEP 8 Compliance</u>

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

Prompt:

Refactor the following Python code to comply with PEP 8 style guidelines.
Ensure proper indentation, spacing, naming conventions, and structure.
Return only the improved Python code.

Code:

```
def area_of_rectangle(length, width):
    """Calculate the area of a rectangle."""
    return length * width
print(area_of_rectangle(10, 20))
```

Output:

```
Ass-10.1.py > ...
#Code:
def area_of_rectangle(length, width):
    """Calculate the area of a rectangle."""
    return length * width

print(area_of_rectangle(10, 20))
#Explanation:
#The original code had a function name that was not descriptive and did not follow PEP 8 naming con
#The refactored code uses a more descriptive function name 'area_of_rectangle' and parameter names
#The function also includes a docstring to explain its purpose, which is a good practice in Python

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE        Python
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Pytho
n/Python311/python.exe "c:/Users/vempa/OneDrive/Pictures/Desktop/HCL LAB/AI ASSISTANT CODING/Ass-10.1.py"
200
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Explanation:

The original code had a function name that was not descriptive and did not follow PEP 8 naming conventions. The parameters were also not descriptive.

The refactored code uses a more descriptive function name 'area_of_rectangle' and parameter names 'length' and 'width'.

The function also includes a docstring to explain its purpose, which is a good practice in Python programming. The code is now compliant with PEP 8 style guidelines and is more readable**.**

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
return x*y/100
a=200
```

b=15

print(c(a,b))

Expected Output:

• Python code with descriptive variable names, inline comments, and clear formatting.

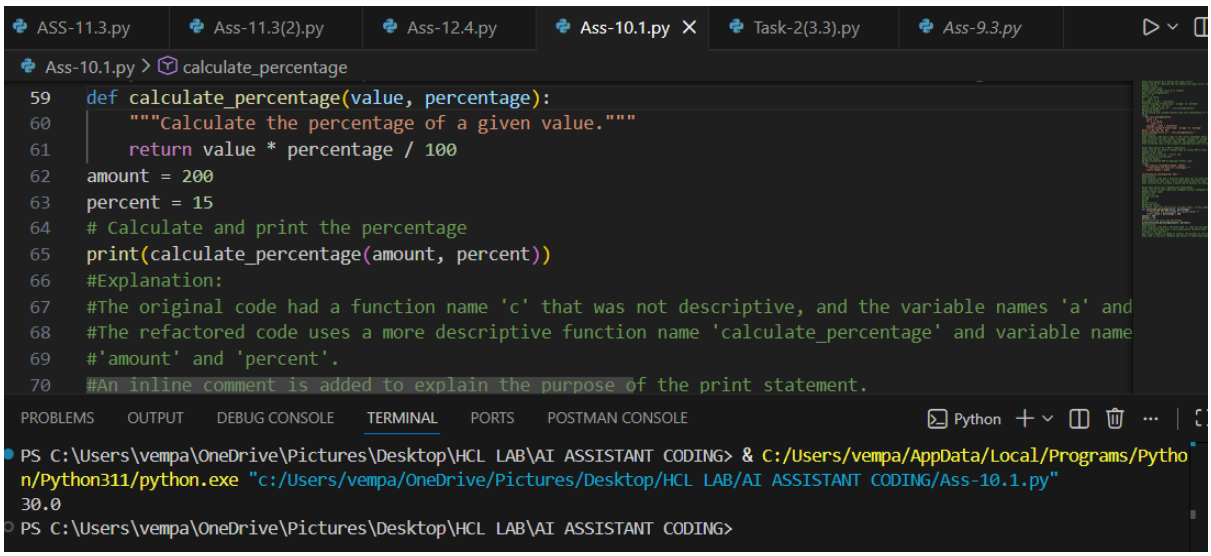
Prompt:

Improve the readability of the following Python code without changing its logic.
Use descriptive variable names, proper formatting, and inline comments where appropriate.
Return the improved Python code.

Code:

```
def calculate_percentage(value, percentage):
    """Calculate the percentage of a given value."""
    return value * percentage / 100
amount = 200
percent = 15
# Calculate and print the percentage
print(calculate_percentage(amount, percent))
```

Output:



Explanation:

The original code had a function name 'c' that was not descriptive, and the variable names 'a' and 'b' were also not descriptive.

The refactored code uses a more descriptive function name 'calculate_percentage' and variable names 'amount' and 'percent'.

An inline comment is added to explain the purpose of the print statement.

The code is now more readable and easier to understand while maintaining the same logic.

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])
```

Expected Output:

• Modular code with reusable functions.

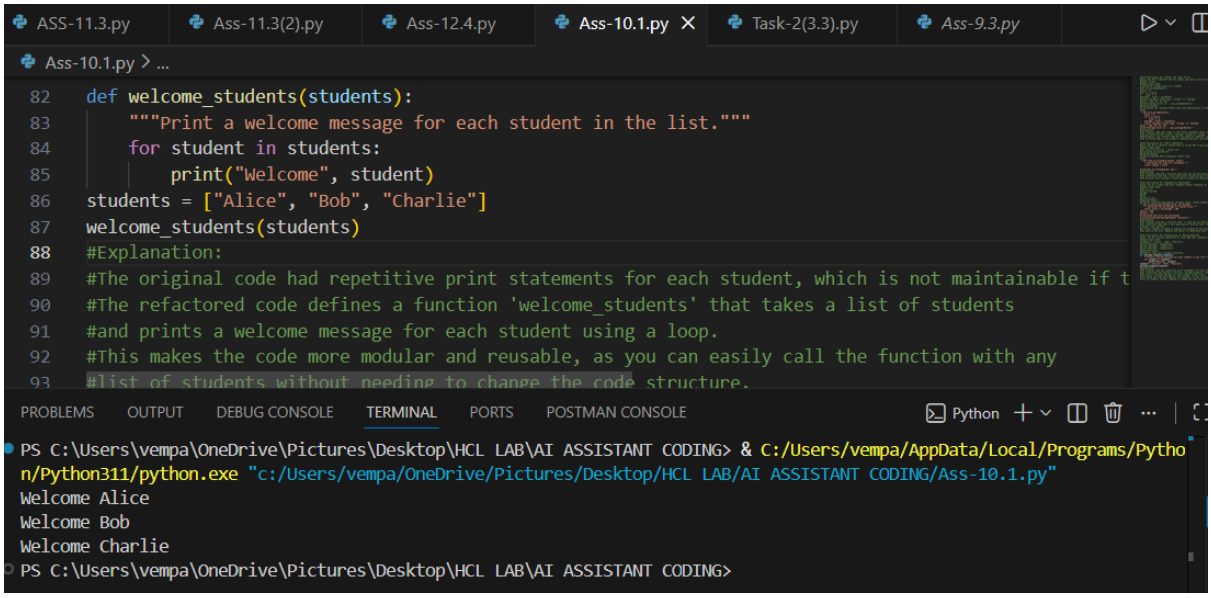
Prompt:

Refactor the following Python code to improve maintainability by removing repetition and creating reusable functions.
Return clean, modular Python code.

Code:

```
def welcome_students(students):

    """Print a welcome message for each student in the list."""

    for student in students:

        print("Welcome", student)

students = ["Alice", "Bob", "Charlie"]

welcome_students(students)
```

Output:



Explanation:

The original code had repetitive print statements for each student, which is not maintainable if the number of students changes.

The refactored code defines a function 'welcome_students' that takes a list of students and prints a welcome message for each student using a loop.

This makes the code more modular and reusable, as you can easily call the function with any list of students without needing to change the code structure.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

squares.append(n**2)

print(len(squares))

Expected Output:

• Optimized code using list comprehensions or vectorized

operations.

Prompt:

Optimize the following Python code to improve performance.
Use efficient techniques such as list comprehensions or built-in functions where appropriate.
Return the optimized Python code.

Code:

import numpy as np

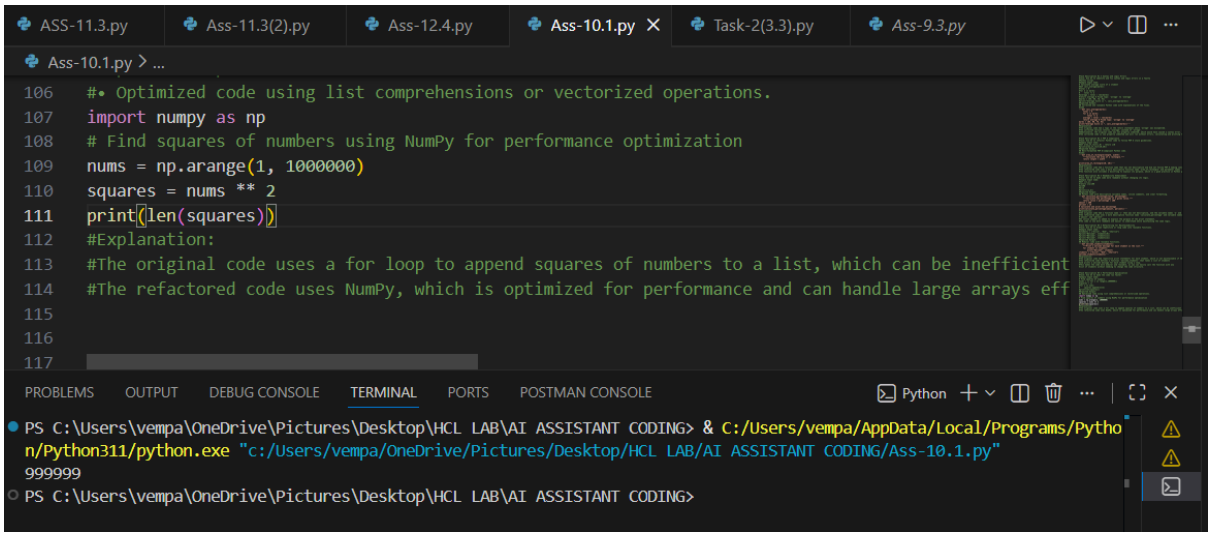
Find squares of numbers using NumPy for performance optimization

nums = np.arange(1, 1000000)

squares = nums ** 2

print(len(squares))

Output:



Explanation:

The original code uses a for loop to append squares of numbers to a list, which can be inefficient for large lists.

The refactored code uses NumPy, which is optimized for performance and can handle large arrays efficiently. By using NumPy's array operations, we can calculate the squares of numbers in a vectorized manner, which is much faster than using a loop.

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

def grade(score):

if score >= 90:

return "A"

```
else:
if score >= 80:
return "B"
else:
if score >= 70:
return "C"
else:
if score >= 60:
return "D"
else:
return "F"
```

Expected Output:

• Cleaner logic using elif or dictionary mapping.

Prompt:

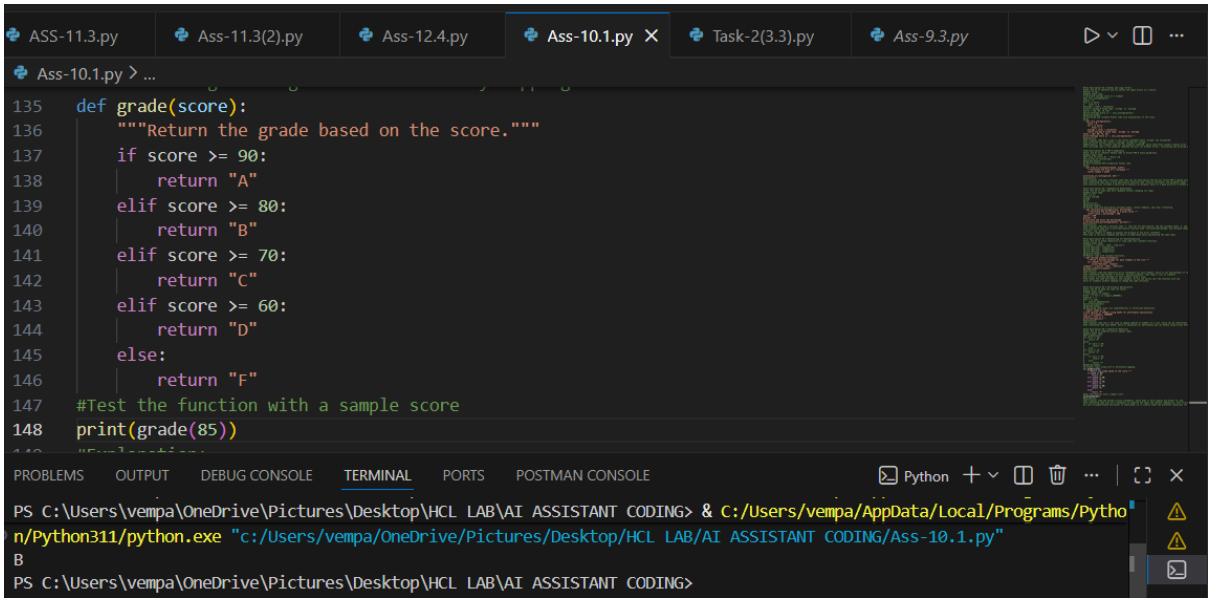
Simplify the following Python function to reduce complexity and improve readability.
Replace nested conditionals with cleaner logic (e.g., using elif or other structured approaches).
Return the improved Python code.

Code:

```
def grade(score):
    """Return the grade based on the score."""
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
```

```
    return "F"

#Test the function with a sample score

print(grade(85))
```

Output:



Explanation:

The original code had nested if-else statements, which made it more complex and harder to read.

The refactored code uses elif statements to simplify the logic and improve readability. This structure is more straightforward and easier to understand, as it clearly shows the different conditions for each grade without unnecessary nesting.